

Text as Data

Peter Dodd
2308057D

Coursework

Notes:

- *The abbreviations M.A and W.A mean Macro Averaged and Weighted Average respectively*
- *In question 1, the W.A results are used in the table, with an added M.A F1 score for selecting the best performing classifier; it was unclear what exactly to do from the coursework specification*
- *I completed this coursework locally on my laptop using jupyter-notebook*

Link to Notebook:

https://drive.google.com/file/d/1jjZt_2ijpMLB3G-DWxIC6-9TKn9G_pY3/view?usp=sharing

Part A Sentiment Polarity Classification

Dataset Overview

All of the datasets have a similar distribution of sentiment polarity. Each has about 1% very negative posts, 7% negative, 63% neutral, 27% positive, and 2% very positive.

This distribution shows that there is a moderate to extreme class imbalance issue. As neutral is a majority class, each of the classifiers will score highest for this class, and thus disproportionately make predictions for this. The scores of the positive class will also probably be higher than the rest as it is the second most populous class. The rest will score poorly, as there is not enough data to train the classifiers effectively. This will cause the overall scores for each classifier to be significantly capped.

Theoretically, undersampling could be applied so that all the classes are balanced. However, this would very likely lead to underfitting, as each class size for the training data would only be around 100. For this reason I decided to not undersample, as it would allow me to use a wider range of classifiers more effectively.

Oversampling is also a possibility to help combat this, but it can lead to faulty classifiers, and is outside the scope of the coursework.

1.i Classifier Results

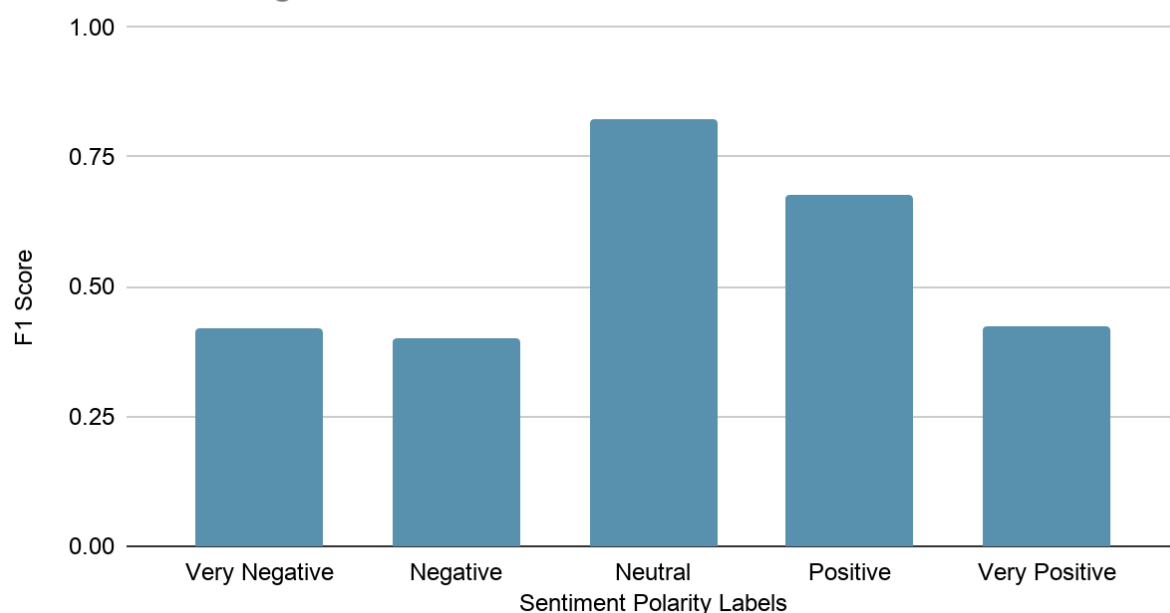
Each of the classifiers were trained using the training data. The results for each of the classifiers predicting the test data and the train data are as follows:

Classifier	Accuracy		W.A Precision		W.A Recall		W.A F1		M.A F1
	Test	Train	Test	Train	Test	Train	Test	Train	Test
a) Dummy Most Frequent	0.636	0.633	0.472	1.000	0.472	0.633	0.472	0.775	0.154
b) Dummy Stratified	0.479	0.464	0.472	0.472	0.472	0.474	0.472	0.473	0.193
c) Logistic Regression One-Hot	0.748	0.963	0.786	0.967	0.748	0.963	0.762	0.963	0.486
d) Logistic Regression TF-IDF	0.739	0.817	0.853	0.903	0.739	0.817	0.778	0.845	0.349
e) SVC One-Hot	0.730	0.862	0.876	0.931	0.730	0.862	0.783	0.887	0.287
f) Neural Network Bag of Words	0.747	0.999	0.765	0.999	0.747	0.999	0.754	0.999	0.549

best M.A F1 score highlighted in yellow

The best performing classifier in terms of test set M.A F1s (neural network from part f) M.A F1 score on the test set for each class:

Best Performing Classifier - Class F1 Scores



1.ii Analysis and Discussion

The self-defined classifier I chose was a neural network classifier with bag-of-words vectorisation. I chose a neural network because it is a popular choice of classifier, the training dataset was large enough for it to be effective, and I had an interest in them and wished to try it with a meaningful dataset. I thought bag-of-words would be an effective choice of vectorisation as the frequency in which terms occur could further signify a posts sentiment. I chose to keep the same vectorizer as used in the one-hot encodings.

These proved to be effective choices, as the neural network was the best performer by a fairly significant margin (in terms of M.A F1). Considering the imbalance in the dataset, the model showed to be a relatively good fit for the dataset. This is shown by the following normalised confusion matrix, with false predictions in **red**, true predictions in **green**, and deeper colours for larger values:

<i>true\predicted</i>	<i>very negative</i>	<i>negative</i>	<i>neutral</i>	<i>positive</i>	<i>very positive</i>
<i>very negative</i>	0.375	0.156	0.469	0.000	0.000
<i>negative</i>	0.011	0.337	0.624	0.028	0.000
<i>neutral</i>	0.004	0.033	0.850	0.108	0.005
<i>positive</i>	0.001	0.008	0.312	0.657	0.022
<i>very positive</i>	0.000	0.000	0.256	0.360	0.384

Although there was an overall tendency to make false predictions for the neutral class, the classifier did generally well as it made more predictions on true classes than any other for each class. This is especially impressive for the 'very positive' and 'very negative' classes, which had considerably smaller sample sizes in the training set. However, it did not manage to make a majority of true predictions for most classes except 'neutral' and 'positive' (the classes making up 90% of the data). It is also clear that there was some amount of overfitting to the training set, as it scored an accuracy of 0.999 on it compared to 0.747 on the test set.

Excluding the self-defined neural network, the best performing classifier (in terms of M.A F1 score) was the logistic regression classifier with one-hot vectorisation, outperforming its TF-IDF counterpart. Both classifiers had default parameters set, apart from max iterations which was increased to 1000 to allow for convergence. The TF-IDF vectorizer had default settings as requested, and the one-hot vectorizer only had max_features set and the tokenizer set to the spaCy based function.

The one-hot vectorized logistic regression classifier showed to be a better fit than the TF-IDF model. This could be because the default tokenizer in the TF-IDF vectorizer is not as effective as the one used in the one-hot encoding. Although this may be true, it is also likely that term frequency throughout the corpus is not actually as important for classifying in this scenario as the term just being there or not. This could make some amount of sense, as a term's presence

could be seen as more important to its classification than its frequency in this context. Both models performed better than the baseline models, showing that the classification methods were somewhat effective.

Excluding the baseline dummy classifiers, the worst performing classifier (in terms of M.A F1 score) was the support vector classifier with default parameters. This is somewhat expected, as support vector machines don't always scale to larger datasets, and they don't work as effectively or efficiently on multiclass problems. The difference in M.A F1 to the baseline models is marginal in comparison to those from the logistic regression and neural network classifiers.

However, when comparing the classifiers W.A F1 on the test dataset, the SVC suddenly comes out on top. This means that taking into account the class imbalance by weighting the results, it performs the best. This is a surprise as SVMs usually greatly benefit from parameter tuning and they don't function as well on multi-class scenarios or larger datasets.

All classifiers performed better than both the baselines. This is indicative of them being at least somewhat effective, as the baselines are very poor methods for classification. One classifies based on the most frequent class, and the other on complete randomness, so it would be rather worrying if either came close to surpassing any of the others in performance.

An interesting point to mention is that all classifiers performed better on the train data set than the test data set. As mentioned before, this can show that the classifiers have been overfit to the training data, and may not be as effective in real world scenarios.

2.i Parameter Tuning

For the parameter tuning, I opted to use a cross-validation grid search. My chosen parameter was the 'solver' for the logistic regression classifier.

I ran three iterations of cross validation to hone in on the best choice of parameters. On the first, I used a wide range of values for each parameter. From this, I found that the 'lbfgs' solver was most effective, as well as 'sublinear_tf' equalling true on the TF-IDF vectorization. Using this, I honed further in on values for 'C' and 'max_features', utilising the results found from the first iteration to limit the parameters for 'solver' and 'sublinear_tf'. This vastly increased performance of the grid search for further iterations.

When at the third iteration, the 'Best Score' output had only seen an increase of around 0.003, so I stopped at this point. The parameters for the final iteration were a 'C' value of 500, the 'solver' set to 'lbfgs', a 'max_features' value of 6000, and 'sublinear_tf' set to true.

The first round of cross validation proved to be the most helpful, as it had the largest range of parameter inputs. Further iterations had only marginal improvements as the parameter ranges decreased. The 'solver' seemed to have the least effect, as it went with the default solver on the first iteration. The 'C' parameter seemed to have the greatest effect. This is because of the effect it has on regularization, and thus on reducing the overfitting problem.

The results for this compared to the baseline logistic regression classifier with TF-IDF vectorization on the test dataset is as follows:

<i>Classifier</i>	<i>Accuracy</i>	<i>M.A Precision</i>	<i>M.A Recall</i>	<i>M.A F1</i>
Baseline LR TF-IDF	0.739	0.320	0.564	0.344
Parameter Tuned LR TF-IDF	0.744	0.498	0.563	0.524

As can be seen from the table, there was a significant jump in M.A F1 score of 0.18, highlighting the importance of tuning to provide well fitted models.

2.ii Error Analysis

For starting error analysis, I wrote the body and sentiment polarity columns as well as the predicted sentiment polarity to a file for examining. Immediately I saw some glaring issues with labelling of the data. There were a lot of mislabelled posts, which would more than likely extend to all datasets used, causing training error. For instance, there was a post that stated “You suffer then die” - this was labelled as neutral, and predicted as neutral. Another post read “There are more Juggalos than polar bears. Edit: Juggalos are die hard Insane Clown Posse fans “ - this was labelled as negative, and predicted as very negative, when it is clear this is far more of a neutral statement than the former. There is labelling like this throughout the dataset, making it near impossible to produce a model which is replicable in the real world.

Beyond that, a normalised confusion matrix can be used to visualise how effective the model is at predicting, and how many mistakes it makes. The matrix contains false predictions in **red**, true predictions in **green**, and deeper colours for larger values:

<i>true\predicted</i>	<i>very negative</i>	<i>negative</i>	<i>neutral</i>	<i>positive</i>	<i>very positive</i>
<i>very negative</i>	0.344	0.219	0.375	0.063	0.000
<i>negative</i>	0.014	0.337	0.610	0.032	0.007
<i>neutral</i>	0.003	0.037	0.831	0.121	0.007
<i>positive</i>	0.000	0.005	0.281	0.697	0.017
<i>very positive</i>	0.000	0.000	0.198	0.523	0.279

With the colour saturation representing a higher proportion of predictions for that class, it is very clear to see that the model has a tendency to class items as neutral, even if they are not. For instance, the negative and very negative classes have more predictions as neutral than they do as negative. As mentioned earlier, this is mostly because of the class imbalance of the datasets, but the mislabelling of will certainly be another contributing factor.

3 Feature Adding

For trying to improve the tuned model, I added the following two features:

- Body w/ One-Hot Vectorizer - The reason I chose this as a feature is because the one-hot vectorizer had generally performed well throughout the classifier shootout so far
- Majority Type w/ One-Hot Vectorizer - I chose this feature as well as I felt that there may be a correlation between sentiment and the type of post (ex. humour is likely positive) and I chose one-hot vectorization as the frequency of terms is irrelevant in this case

The results on the test dataset compared to the classifier from question 2 are as follows:

<i>Classifier</i>	<i>Accuracy</i>	<i>M.A Precision</i>	<i>M.A Recall</i>	<i>M.A F1</i>
Tuned LR TF-IDF	0.744	0.498	0.563	0.524
Tuned LR TF-IDF w/ Added Features	0.740	0.539	0.543	0.541

The normalized confusion matrix for the classifier is as follows, with false predictions in **red**, true predictions in **green**, and deeper colours for larger values:

<i>true\predicted</i>	<i>very negative</i>	<i>negative</i>	<i>neutral</i>	<i>positive</i>	<i>very positive</i>
<i>very negative</i>	0.406	0.219	0.375	0.000	0.000
<i>negative</i>	0.021	0.418	0.528	0.028	0.004
<i>neutral</i>	0.005	0.051	0.813	0.122	0.010
<i>positive</i>	0.001	0.008	0.268	0.696	0.027
<i>very positive</i>	0.000	0.012	0.163	0.465	0.360

The added features had a marginally positive effect on the classifier, with an overall increase of 0.017 for it's M.A F1 score. What is interesting, however, is the decreases in both accuracy and M.A recall. This means that less documents were classified correctly, and that on average, less correct predictions were made in each class. The increased M.A precision meant that the model on average labelled less samples incorrectly.

The overall increase is indicative that the features added are relevant and useful to the model. The features added do not create a significant increase however, and may create an overly complex model. To help combat this, it would be more effective to run parameter tuning after adding features. This way, regularisation parameters might help best make use of the new features.

Part B Thread Subreddit Classification

4.i Classifier Results

On top of the baseline logistic regression classifier, I chose to train a SVC, a neural network, and a decision tree. I firstly trained each without optimisation or cross-validation. The results for each of these on the test dataset are as follows:

<i>Classifier</i>	<i>Accuracy</i>	<i>M.A Precision</i>	<i>M.A Recall</i>	<i>M.A F1</i>
Logistic Regression	0.592	0.444	0.622	0.472
SVC	0.553	0.341	0.651	0.358
Neural Network	0.614	0.484	0.601	0.500
Decision Tree	0.416	0.279	0.307	0.277

Compared to the baseline, both the geometric classifiers performed significantly worse. This could be because of the sheer amount of classes in the dataset. The neural network outperformed all of them. This is expected, as they tend to be more powerful and effective at handling multiclass classification.

4.ii Feature Adding and Parameter Optimisation

I have added the following two features to these classifiers:

- Bigrams - To help add more context within a post
- Sentiment Polarity - Posts sentiment may be indicative of the material posted in a certain subreddit

I have also run parameter optimisation on all of the classifiers. I opted to just tune classifier parameters, as I didn't want the processing overhead to be too large. I focused the tuning on trying different solvers, kernels and splitters to try and make the biggest impact. The results on the test datasets with the optimisation and added features are as follows:

<i>Classifier</i>	<i>Accuracy</i>	<i>M.A Precision</i>	<i>M.A Recall</i>	<i>M.A F1</i>
Logistic Regression	0.586	0.438	0.542	0.458
SVC	0.538	0.354	0.561	0.379
Neural Network	0.598	0.458	0.603	0.485
Decision Tree	0.452	0.309	0.345	0.309

The results after optimisation are surprising. Both the logistic regression and neural network classifiers see decreases in M.A F1, whilst the geometric classifiers both see a slight improvement.

This could be indicative of a poor choice of features. Sentiment polarity may be too vague to be able to attribute to particular classes, especially since the dataset has an imbalance towards the 'neutral' class. Bigrams, however, surprisingly don't have a positive impact. It is possible that this is because the parameters on it have not been tuned, or that the type of vectorization used for it is not as effective as another would've been, possibly a bag of words encoding.